Taller 3 - Tres APIs Flask Balanceadas con PostgreSQL

Workshop 3

Parte 1:

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres/db$ cat init.sql
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) NOT NULL
);

INSERT INTO products (name, price) VALUES
('Laptop', 3500000),
('Mouse', 80000),
('Teclado', 150000),
('Monitor', 900000),
('Audifonos', 200000);
```

Parte 2:

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres/api$ cat requirements.txt
Flask
psycopg[binary]
juanc@JHANKPC:~/cloud/workshop-flask-postgres/api$ cat app.py
import os

import psycopg
from flask import Flask, jsonify, request

app = Flask(__name__)


def get_connection():
    return psycopg.connect(
        host=os.getenv("DB_HOST"),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD")
    )


@app.route("/products", methods=["GET"])
def get_products():
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT id, name, price FROM products ORDER BY id")
            rows = cur.fetchall()
```

```
@app.route("/products", methods=["GET"])
def get_products():
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT id, name, price FROM products ORDER BY id")
            rows = cur.fetchall()

    products = []

    for row in rows:
        products.append({
            "id": row[0],
            "name": row[1],
            "price": float(row[2])
        })

    return jsonify({
        "served_by": os.getenv("INSTANCE"),
        "products": products
    })


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
juanc@JHANKPC:~/cloud/workshop-flask-postgres/api$
```

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres/api$ cat Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Parte 3:

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ cat docker-compose.yml
services:

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

  api1:
    build: ./api
    environment:
      INSTANCE: api-1
      DB_HOST: db
      DB_NAME: shop
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  api2:
    build: ./api
    environment:
      INSTANCE: api-2
      DB_HOST: db
```

```
      DB_NAME: shop
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  api3:
    build: ./api
    environment:
      INSTANCE: api-3
      DB_HOST: db
      DB_NAME: shop
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - api1
      - api2
      - api3

volumes:
  pgdata:
```

Parte 4:

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ cat nginx.conf
events {}

http {
    upstream flask_servers {
        server api1:5000;
        server api2:5000;
        server api3:5000;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://flask_servers;

            proxy_next_upstream error timeout http_502 http_503 http_504;
        }
    }
}
```

Parte 5:

```
[+] up 27/27esolving provenance for metadata file
 ✔Image postgres:16                          Pulled
 ✔Image workshop-flask-postgres-api2         Built
 ✔Image workshop-flask-postgres-api3         Built
 ✔Image workshop-flask-postgres-api1         Built
 ✔Network workshop-flask-postgres_default    Created
 ✔Volume workshop-flask-postgres_pgdata      Created
 ✔Container workshop-flask-postgres-db-1     Created
 ✔Container workshop-flask-postgres-api1-1   Created
 ✔Container workshop-flask-postgres-api2-1   Created
 ✔Container workshop-flask-postgres-api3-1   Created
 ✔Container workshop-flask-postgres-nginx-1  Created
Attaching to api1-1, api2-1, api3-1, db-1, nginx-1
db-1  | The files belonging to this database system will be owned by user "postgres".
db-1  | This user must also own the server process.
db-1  |
db-1  | The database cluster will be initialized with locale "en_US.utf8".
db-1  | The default database encoding has accordingly been set to "UTF8".
db-1  | The default text search configuration will be set to "english".
db-1  |
db-1  | Data page checksums are disabled.
db-1  |
db-1  | fixing permissions on existing directory /var/lib/postgresql/data ... ok
db-1  | creating subdirectories ... ok
db-1  | selecting dynamic shared memory implementation ... posix
db-1  | selecting default max_connections ... 100
db-1  | selecting default shared_buffers ... 128MB
db-1  | selecting default time zone ... Etc/UTC
```

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ docker compose ps
NAME                              IMAGE                          COMMAND                  SERVICE   CREATED         STAT
US          PORTS
workshop-flask-postgres-api1-1    workshop-flask-postgres-api1   "python app.py"          api1      2 minutes ago   Up 2
 minutes    5000/tcp
workshop-flask-postgres-api2-1    workshop-flask-postgres-api2   "python app.py"          api2      2 minutes ago   Up 2
 minutes    5000/tcp
workshop-flask-postgres-api3-1    workshop-flask-postgres-api3   "python app.py"          api3      2 minutes ago   Up 2
 minutes    5000/tcp
workshop-flask-postgres-db-1      postgres:16                    "docker-entrypoint.s…"   db        2 minutes ago   Up 2
 minutes    5432/tcp
workshop-flask-postgres-nginx-1   nginx:alpine                   "/docker-entrypoint.…"   nginx     2 minutes ago   Up 2
 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ curl http://localhost:8080/products
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-1"}
juanc@JHANKPC:~/cloud/workshop-flask-postgres$
```

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ for i in {1..15}; do curl -s http://localhost:8080/products; echo; done
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
```

Parte 6:

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ docker compose stop api2
[+] stop 1/1
 ✔Container workshop-flask-postgres-api2-1 Stopped                                                                  1.4s
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ docker compose ps
NAME                                IMAGE                          COMMAND                  SERVICE   CREATED          STAT
US          PORTS
workshop-flask-postgres-api1-1      workshop-flask-postgres-api1   "python app.py"          api1      5 minutes ago    Up 5
 minutes    5000/tcp
workshop-flask-postgres-api3-1      workshop-flask-postgres-api3   "python app.py"          api3      5 minutes ago    Up 5
 minutes    5000/tcp
workshop-flask-postgres-db-1        postgres:16                    "docker-entrypoint.s…"   db        5 minutes ago    Up 5
 minutes    5432/tcp
workshop-flask-postgres-nginx-1     nginx:alpine                   "/docker-entrypoint.…"   nginx     5 minutes ago    Up 5
 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
juanc@JHANKPC:~/cloud/workshop-flask-postgres$
```

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres$ for i in {1..15}; do curl -s http://localhost:8080/products; echo; done
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0}],"served_by":
"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
```

No hay solicitudes de api-2

Parte 7:

```
  GNU nano 8.7.1                                                app.py
import os
import psycopg

from flask import Flask, jsonify, request

app = Flask(__name__)


def get_connection():
    return psycopg.connect(
        host=os.getenv("DB_HOST"),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD")
    )


# GET /products
@app.route("/products", methods=["GET"])
def get_products():

    conn = get_connection()
    cur = conn.cursor()

    cur.execute("SELECT id, name, price FROM products ORDER BY id")
    rows = cur.fetchall()
```

```python
# GET /products/<id>
@app.route("/products/<int:id>", methods=["GET"])
def get_product(id):

    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        "SELECT id, name, price FROM products WHERE id = %s",
        (id,)
    )

    row = cur.fetchone()

    cur.close()
    conn.close()

    if row is None:
        return jsonify({"error": "Producto no encontrado"}), 404

    return jsonify({
        "served_by": os.getenv("INSTANCE"),
        "product": {
            "id": row[0],
            "name": row[1],
            "price": float(row[2])
```

```python
# DELETE /products/<id>
@app.route("/products/<int:id>", methods=["DELETE"])
def delete_product(id):

    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        "DELETE FROM products WHERE id = %s",
        (id,)
    )

    conn.commit()

    cur.close()
    conn.close()

    return jsonify({
        "served_by": os.getenv("INSTANCE"),
        "message": "Producto eliminado"
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres/api$ docker compose ps
NAME                              IMAGE                         COMMAND                  SERVICE   CREATED          STA
TUS          PORTS
workshop-flask-postgres-api1-1    workshop-flask-postgres-api1   "python app.py"          api1      51 seconds ago   Up
47 seconds   5000/tcp
workshop-flask-postgres-api2-1    workshop-flask-postgres-api2   "python app.py"          api2      51 seconds ago   Up
47 seconds   5000/tcp
workshop-flask-postgres-api3-1    workshop-flask-postgres-api3   "python app.py"          api3      51 seconds ago   Up
47 seconds   5000/tcp
workshop-flask-postgres-db-1      postgres:16                    "docker-entrypoint.s…"   db        13 minutes ago   Up
13 minutes   5432/tcp
workshop-flask-postgres-nginx-1   nginx:alpine                   "/docker-entrypoint.…"   nginx     13 minutes ago   Up
13 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
juanc@JHANKPC:~/cloud/workshop-flask-postgres/api$ curl -X POST http://localhost:8080/products \
> -H "Content-Type: application/json" \
> -d '{"name":"ProductoPrueba","price":50000}'
{"message":"Producto creado","served_by":"api-1"}
```

```
juanc@JHANKPC:~/cloud/workshop-flask-postgres/api$ for i in {1..6}; do curl -s http://localhost:8080/products; echo; don
e
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0},{"id":6,"name
":"ProductoPrueba","price":50000.0}],"served_by":"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0},{"id":6,"name
":"ProductoPrueba","price":50000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0},{"id":6,"name
":"ProductoPrueba","price":50000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0},{"id":6,"name
":"ProductoPrueba","price":50000.0}],"served_by":"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0},{"id":6,"name
":"ProductoPrueba","price":50000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado"
,"price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":200000.0},{"id":6,"name
":"ProductoPrueba","price":50000.0}],"served_by":"api-3"}
```

Análisis

1. ¿Dónde se encuentra el estado del negocio?
   En PostgreSQL, porque allí se almacenan los productos y sus datos.
2. ¿Por qué las APIs pueden ser stateless?
   Porque no guardan datos localmente; solo procesan solicitudes y consultan PostgreSQL.
3. ¿Qué ocurre si una API reinicia?
   No se pierden los datos y, al volver a iniciar, puede seguir consultando la misma base de datos.
4. ¿Qué ocurre si PostgreSQL reinicia?
   Las APIs no pueden consultar datos mientras esté caído. Si se usa un volumen, los datos se conservan al reiniciar.
5. ¿Dónde existen puntos únicos de falla?
   Principalmente en Nginx y PostgreSQL, porque solo hay una instancia de cada uno.

6. ¿El balanceamiento de API soluciona el escalamiento de la base de datos? No. Solo distribuye las solicitudes entre las APIs; PostgreSQL sigue siendo una única base de datos.
7. ¿Qué mecanismos podrían añadirse en una arquitectura de producción? Réplicas, backups, monitoreo, HTTPS, gestión segura de credenciales y redundancia de servicios.